

# Unit 3 · Topic Notes

## CRUD Operations with Models and Controllers

**Course:** Full Stack Application Development with MERN

### Learning Objectives

1. Map CRUD operations to HTTP methods, routes and Mongoose queries
2. Write controllers that return the correct status codes
3. Add pagination, filtering, sorting and search safely
4. Handle errors in one central place
5. Recognise and handle the edge cases that break real CRUD APIs

### 1. CRUD ↔ HTTP ↔ Mongoose

| Operation | HTTP        | Route             | Mongoose                       | Success code |
|-----------|-------------|-------------------|--------------------------------|--------------|
| Create    | POST        | /api/products     | Product.create(data)           | 201 Created  |
| Read all  | GET         | /api/products     | Product.find(filter)           | 200 OK       |
| Read one  | GET         | /api/products/:id | Product.findById(id)           | 200 OK       |
| Update    | PATCH / PUT | /api/products/:id | Product.findByIdAndUpdate(...) | 200 OK       |
| Delete    | DELETE      | /api/products/:id | Product.findByIdAndDelete(id)  | 200 or 204   |

Common error codes: 400 bad input, 401 not logged in, 403 not allowed, 404 not found, 409 conflict (duplicate), 500 server bug.

### 2. Routes (thin) and Controllers (the logic)

```
// routes/productRoutes.js
import express from "express";
import {
  createProduct, getProducts, getProduct, updateProduct, deleteProduct,
} from "../controllers/productController.js";
import { validateId } from "../middleware/validateId.js";

const router = express.Router();

router.route("/").get(getProducts).post(createProduct);
router.route("/:id").all(validateId).get(getProduct).patch(updateProduct).delete(deleteProduct);

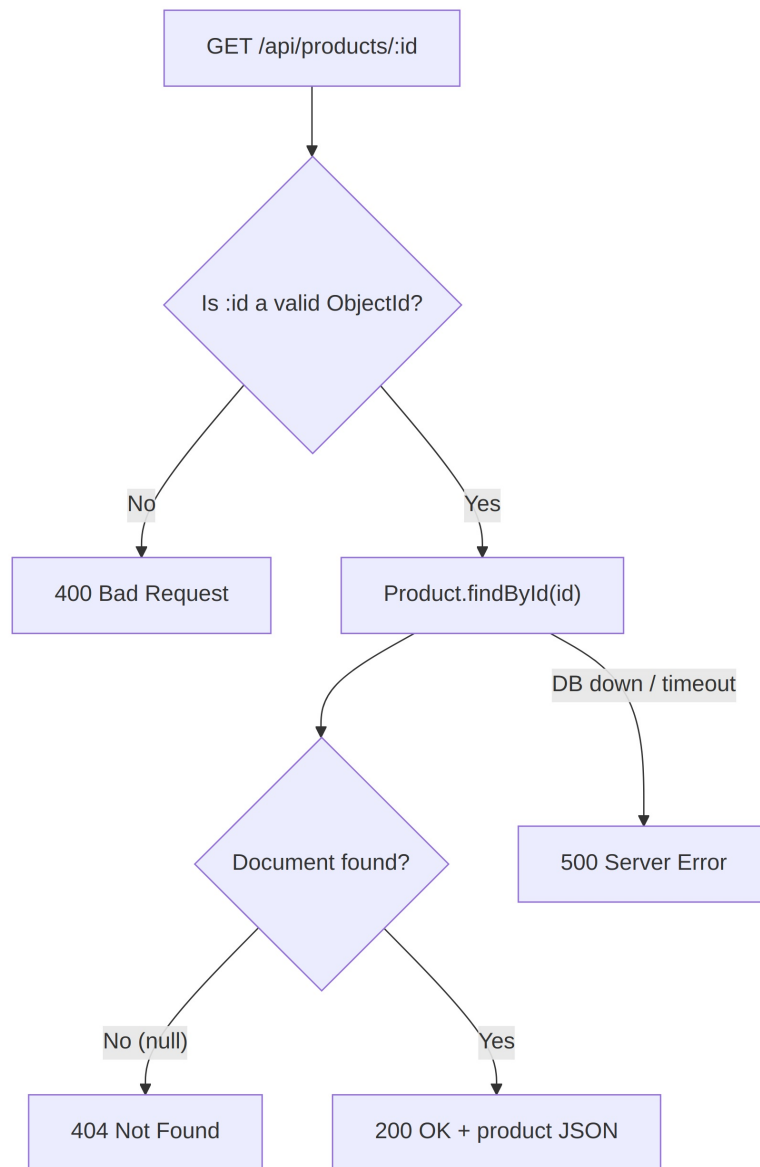
export default router;
```

```
// server.js (relevant lines)
import productRoutes from "../routes/productRoutes.js";
import { notFound, errorHandler } from "../middleware/errorMiddleware.js";

app.use("/api/products", productRoutes);
app.use(notFound); // unknown routes -> 404 JSON
app.use(errorHandler); // must be LAST
```

**Express version note:** In Express 5 (the current default from `npm install express`), a rejected promise inside an `async` handler is passed to your error middleware automatically. In Express 4 you must wrap handlers in `try/catch` and call `next(err)`, or use a wrapper such as `asyncHandler`. The controllers below rely on the Express 5 behaviour.

### 3. Guarding the `:id` Parameter



Read-one decision flow

```
// middleware/validateId.js
const OBJECT_ID = /^[a-f\d]{24}$/i;

export function validateId(req, res, next) {
  if (!OBJECT_ID.test(req.params.id)) {
    return res.status(400).json({ message: "Invalid id format" });
  }
  next();
}
```

**Edge case:** `mongoose.isValidObjectId("abcdefghijkl")` returns `true` because any 12-character string can be cast to an ObjectId. The 24-character hex regex above is stricter and safer.

**Two different “not found” situations:**

- `/api/products/123` → the id is malformed → `400` (without the guard, Mongoose throws a `CastError` → would become a `500`).
- `/api/products/64b7f0c2a1b2c3d4e5f60789` → valid format, no such document → `findById` returns `null` → `404`.

## 4. The Controllers

### 4.1 Create

```
// controllers/productController.js
import Product from "../models/Product.js";

export const createProduct = async (req, res) => {
  const { name, price, stock, category, sku } = req.body; // whitelist fields
  const product = await Product.create({ name, price, stock, category, sku });
  res.status(201).json(product);
};
```

**Why whitelist?** If you wrote `Product.create(req.body)`, a client could send `{"isActive": false, "createdAt": "2000-01-01"}` or, on a User model, `{"role": "admin"}`. This is called **mass assignment**. Always pick the fields you accept.

### 4.2 Read all: pagination, filter, sort, search

```
const escapeRegex = (s) => s.replace(/[\.*?^${}()|[\]\$\\"/g, "\\$&");

export const getProducts = async (req, res) => {
  // --- pagination with safe limits ---
  const page = Math.max(parseInt(req.query.page) || 1, 1); // NaN or 0 -> 1
  const limit = Math.min(Math.max(parseInt(req.query.limit) || 10, 1), 50); // 1..50

  // --- filters (only accept plain strings) ---
  const filter = { isActive: true };
  if (typeof req.query.category === "string") filter.category = req.query.category;

  const toNum = (v) =>
    typeof v === "string" && v.trim() !== "" && !Number.isNaN(Number(v)) ? Number(v) : undefined;
  const min = toNum(req.query.minPrice);
  const max = toNum(req.query.maxPrice);
  if (min !== undefined || max !== undefined) {
    filter.price = {};
    if (min !== undefined) filter.price.$gte = min;
    if (max !== undefined) filter.price.$lte = max;
  }

  // --- text search, regex characters escaped ---
  if (typeof req.query.q === "string" && req.query.q.trim()) {
    filter.name = { $regex: escapeRegex(req.query.q.trim()), $options: "i" };
  }

  // --- sorting from an allow-list ---
  const sortMap = { price_asc: { price: 1 }, price_desc: { price: -1 }, newest: { createdAt: -1 } };
  const sort = sortMap[req.query.sort] || sortMap.newest;

  const [items, total] = await Promise.all([
    Product.find(filter).sort(sort).skip((page - 1) * limit).limit(limit).lean(),
    Product.countDocuments(filter),
  ]);

  res.json({ items, page, limit, total, pages: Math.ceil(total / limit) });
};
```

Try it: `GET /api/products?category=books&minPrice=50&sort=price_asc&page=2&limit=5&q=note`

### 4.3 Read one

```
export const getProduct = async (req, res) => {
  const product = await Product.findById(req.params.id);
  if (!product) return res.status(404).json({ message: "Product not found" });
  res.json(product);
};
```

## 4.4 Update

```
export const updateProduct = async (req, res) => {
  const allowed = ["name", "price", "stock", "category", "isActive"];
  const updates = Object.fromEntries(
    Object.entries(req.body).filter(([key]) => allowed.includes(key))
  );

  if (Object.keys(updates).length === 0) {
    return res.status(400).json({ message: "No valid fields to update" });
  }

  const product = await Product.findByIdAndUpdate(req.params.id, updates, {
    new: true,          // return the UPDATED document
    runValidators: true, // apply schema validation on update too
  });

  if (!product) return res.status(404).json({ message: "Product not found" });
  res.json(product);
};
```

## 4.5 Delete

```
export const deleteProduct = async (req, res) => {
  const product = await Product.findByIdAndDelete(req.params.id);
  if (!product) return res.status(404).json({ message: "Product not found" });
  res.json({ message: "Product deleted", id: product._id });
};
```

---

## 5. Central Error Handling

```
// middleware/errorMiddleware.js
export const notFound = (req, res) => {
  res.status(404).json({ message: `Route not found: ${req.method} ${req.originalUrl}` });
};

export const errorHandler = (err, req, res, next) => {
  // Mongoose schema validation failed
  if (err.name === "ValidationError") {
    const errors = Object.fromEntries(
      Object.entries(err.errors).map(([field, e]) => [field, e.message])
    );
    return res.status(400).json({ message: "Validation failed", errors });
  }
  // Bad type, e.g. price: "abc"
  if (err.name === "CastError") {
    return res.status(400).json({ message: `Invalid value for ${err.path}` });
  }
  // Duplicate unique field
  if (err.code === 11000) {
    const field = Object.keys(err.keyValue)[0];
    return res.status(409).json({ message: `${field} already exists` });
  }
  // Malformed JSON body: {"name": "Pen",}
  if (err.type === "entity.parse.failed") {
    return res.status(400).json({ message: "Malformed JSON in request body" });
  }

  console.error(err); // full detail goes to YOUR logs...
  res.status(500).json({ message: "Something went wrong" }); // ...not to the client
};
```

---

## 6. Edge Cases Checklist

1. `findByIdAndUpdate` returns the OLD document by default. Without `{ new: true }` your response shows the

data from before the update, and students think the update failed.

**2. Update validators are off by default.** Without `runValidators: true`, `PATCH { "price": -100 }` saves happily, bypassing `min: 0`. Note that validators on update only check the fields being updated.

**3. PUT vs PATCH.** `PUT` replaces the whole resource (all fields required); `PATCH` changes only the fields sent. Most apps implement `PATCH` and call it “update”.

**4. Empty or missing body.** If the client forgets the `Content-Type: application/json` header, `req.body` can be `undefined` or `{}`. Destructuring `undefined` crashes, so guard for it (`const { name } = req.body ?? {};`) or validate the body first (next topics).

**5. Deleting twice.** The first `DELETE` returns `200`; the second returns `404`. Some teams prefer returning `204` both times because delete is *idempotent* (repeating it leaves the same final state). Choose one behaviour and document it.

**6. Soft delete for anything referenced elsewhere.** Instead of removing a product that old orders point to, set `isActive: false` and filter it out of list queries.

**7. Pagination abuse.** `?limit=1000000` can pull your whole database into memory. Always cap `limit`. `?page=-3` or `?page=abc` must fall back to safe defaults (done above).

**8. Regex injection in search.** `?q=(a+)+$` or `?q=.*` in a raw `$regex` can be slow or match everything. Escape user input (done above).

**9. Query values may not be strings.** A client can send `?category[$ne]=x`. Depending on the query parser, `req.query.category` may be an object, which becomes a MongoDB operator. Check `typeof value === "string"` before using it in a filter.

**10. Race conditions on counters.** Two people buying the last item at the same time:

```
// BAD: read, modify, save. Both requests read stock = 1.
const p = await Product.findById(id);
if (p.stock >= qty) { p.stock -= qty; await p.save(); } // both succeed -> stock = -1

// GOOD: one atomic, conditional update
const updated = await Product.findOneAndUpdate(
  { _id: id, stock: { $gte: qty } },
  { $inc: { stock: -qty } },
  { new: true }
);
if (!updated) return res.status(409).json({ message: "Insufficient stock" });
```

**11. `.lean()` returns plain objects.** Faster for read-only lists, but you lose virtuals (like `inStock`), getters and `.save()`.

**12. Returning secrets.** When you build a User controller later, never return `password`. Exclude it in the schema (`select: false`) so it can't leak by accident.

---

## 7. Hands-On Lab (60 min, pairs)

---

- Build the full `Product` CRUD API from this note (routes, controllers, error middleware).
  - Test every endpoint in Postman or Thunder Client, including these failure cases and the status code you expect:
    - `GET /api/products/abc`
    - `GET /api/products/64b7f0c2a1b2c3d4e5f60789` (valid format, no such product)
    - `POST` with a negative price
    - `POST` with malformed JSON
    - `POST` twice with the same `sku`
    - `PATCH` with `{ "price": -1 }` (does it get blocked? try with and without `runValidators`)
  - Add pagination and confirm `?limit=99999` is capped at 50.
  - Implement the atomic stock decrement as `POST /api/products/:id/purchase` and fire two requests at once.
  - Commit on a `feature/product-crud` branch and open a PR.
-

## 8. Quick Assessment

---

1. Which HTTP method and status code do you use to create a resource?
  2. What is the difference between a `400` and a `404` for `/api/products/:id`?
  3. Why is `Product.create(req.body)` dangerous?
  4. What two options should you almost always pass to `findByIdAndUpdate`, and why?
  5. Why is “read stock, subtract, save” unsafe under concurrent requests?
  6. Where should `ValidationError` and duplicate-key errors be handled, and why there?
- 

## 9. Common Mistakes to Avoid

---

- Sending `200` for everything, or `500` for user mistakes
  - Putting `try/catch` and error formatting copy-pasted into every controller
  - Returning the raw `err.message` from the database to the client
  - Forgetting `app.use(errorHandler)` **after** the routes (order matters)
  - No cap on list size
- 

*Notes prepared for Full Stack Learning — MERN course, Unit 3.*